Davante

10

PROGRAMACIÓN

Herencia

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Concepto de herencia 5

/ 3. Tipos de herencia 6

3.1. Herencia simple 6

3.2. Herencia múltiple 7

/ 4. Caso práctico 1: "Hasta dónde llegar con la herencia" 8

/ 5. Modificador de acceso para herencia 9

/ 6. Sobreescritura de miembros heredados 10

/ 7. Instrucción super 11

7.1. Llamada al constructor de la clase padre 11

7.2. Acceso a variables y métodos 12

/ 8. Selección dinámica de métodos: Polimorfismo 12

/ 9. Polimorfismo estático 13

/ 10. Polimorfismo dinámico 15

/ 11. Caso práctico 2: "Salario del empleado" 16

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 12. Métodos heredados 17

/ 13. Clases y métodos finales 18

13.1. Clases Finales 18

13.2. Métodos finales 18

13.3. Uso y beneficios 18

/ 14. Resumen y resolución del caso práctico de la unidad 19

/ 15. Bibliografía 20

OBJETIVOS

Conocer el concepto y la necesidad de herencia entre clases.

Comprender la idea de superclase y subclase.

Definir limitaciones de acceso a la superclase.

Utilizar el mecanismo de sobreescritura de métodos.

Usar la selección dinámica de métodos en tiempo de ejecución.

Entender el concepto de clase final y emplearlo.

/ 1. Introducción y contextualización práctica

El concepto de herencia en la programación orientada a objetos (POO) surge de la necesidad de reutilizar y extender el código de manera eficiente, basándose en la idea de que las nuevas clases pueden heredar las propiedades y los comportamientos de clases existentes.

Se trata de una herramienta fundamental para la reutilización de código, la creación de jerarquías de clases y la implementación de polimorfismo.

Aunque puede aumentar la complejidad del código y acoplar estrechamente las clases, sus ventajas en términos de reutilización, mantenimiento y extensibilidad hacen que sea un pilar importante en el desarrollo de aplicaciones.

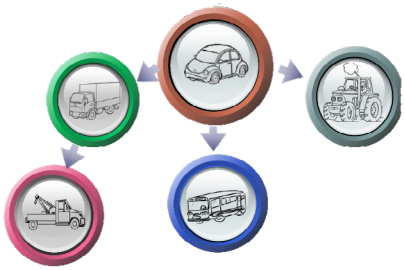

Fig.1. Herencia.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Hipets"

https://on.soundcloud.com/5qwLRALiWx34wm96A

/ 2. Concepto de herencia

La **herencia** es uno de los pilares de la programación orientada a objetos. Gracias a ella vamos a poder reutilizar nuestro código de una manera no vista hasta ahora, reduciendo así la complejidad de nuestros proyectos.

Utilizando la herencia vamos a poder crear clases que se apoyan en otras, elaborando una jerarquía de clases y evitando la recodificación y prueba de muchos métodos.

¡! Sabías que...

En la década de 1960, el desarrollo de lenguajes como Simula y Smalltalk introdujo la idea de clases y objetos, lo que condujo a la conceptualización de la herencia.

Aquí tenemos que distinguir ahora dos conceptos muy importantes:

- **Clase padre o superclase**: Esta es la clase de la que se ha heredado.
- **Clase hija o subclase**: Esta es la clase que hereda de otra.

Cuando una clase hereda de otra, se dice que esta hereda el comportamiento de su clase padre. Esto se refiere a que la clase que hereda, por el mero hecho de heredar, tiene todos los atributos y sus métodos, incluidos constructores, métodos **get**, métodos **set**, el método **toString** y todos los métodos que tenga la clase padre.

Cuando se crea un objeto de la clase hija, se pueden llamar a los métodos de la clase padre, teniendo el mismo comportamiento.

Tenemos que tener cuidado, ya que cuando heredemos de una clase, no podremos ver su parte privada. Si quisiéramos acceder a esa parte, deberemos hacerlo mediante métodos, como hemos estado haciendo hasta ahora normalmente.

Las **ventajas** de la herencia son muchas, entre ellas tenemos:

- Reduce la cantidad de código, ya que no hace falta volver a implementar los métodos heredados.
- No será necesario testear los métodos heredados, ya que se testearon cuando se hicieron en la clase padre.

Cuando una clase hereda de otra, decimos que 'es un' de la clase heredada. Por ejemplo, en la siguiente figura: la clase Perro es un Animal, siendo Animal otra clase.

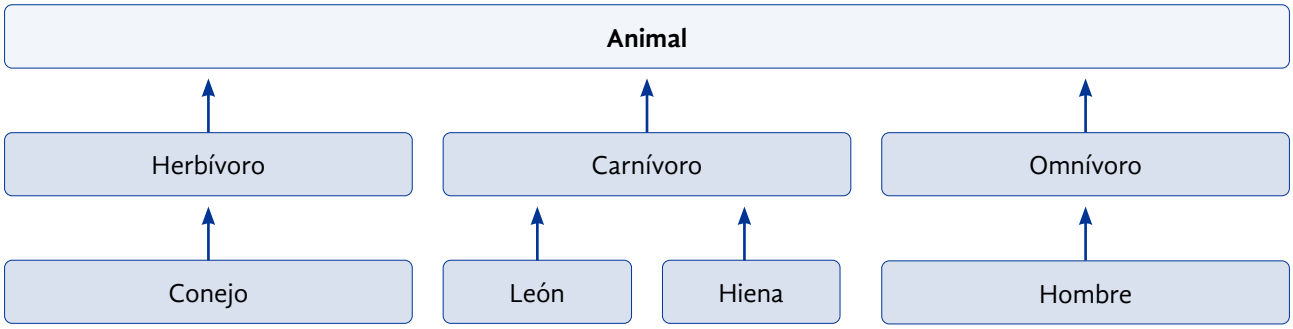

Fig.2. Jerarquía de clases de Animal.

La clave para saber si hay que utilizar la herencia es cuando la clase heredada añade una funcionalidad extra al comportamiento de la clase padre.

/ 3. Tipos de herencia

En programación orientada a objetos existen dos tipos de herencia:

- **Herencia simple**.
- **Herencia múltiple**.

3.1. Herencia simple

En la herencia simple, una clase únicamente va a poder heredar de otra, es decir, **solo va a tener una clase padre**.

Su funcionamiento es en base a la relación entre clases, es decir, cuando una clase hereda de otra, adquiere todo su comportamiento, tanto variables como métodos, incluyéndose también toda variable y método privado que haya en la clase padre.

El concepto de herencia es muy simple, pero muy potente a la vez, ya que si conseguimos aprender a utilizarlo bien, nos simplificará en gran medida nuestros programas.

Ya sabemos que Java es un lenguaje puramente orientado a objetos, es decir, todo son clases, y estas clases deben estar relacionadas entre sí, como es de esperar. Esta relación la tenemos con la clase **Object**.

Toda clase en Java, desde las internas hasta una que podamos crear nosotros mismos, heredan de Object, es decir, que todo son objetos.

Para representar en Java que una clase hereda de otra, utilizamos la palabra reservada **extends**, que va en la declaración de la clase hija, seguida del nombre de la clase de la que va a heredar.

Al ser herencia simple con lo que vamos a trabajar, no podremos poner más de un extends en la declaración de una clase. Lo que sí podemos hacer sin ningún tipo de limitación es que una clase herede de otra, que a su vez hereda de otra.

En el ejemplo mostrado en la siguiente figura: A hereda de B y B hereda de A, aquí tendremos que A es un B, B es un C, y, por lo tanto, A es un C.

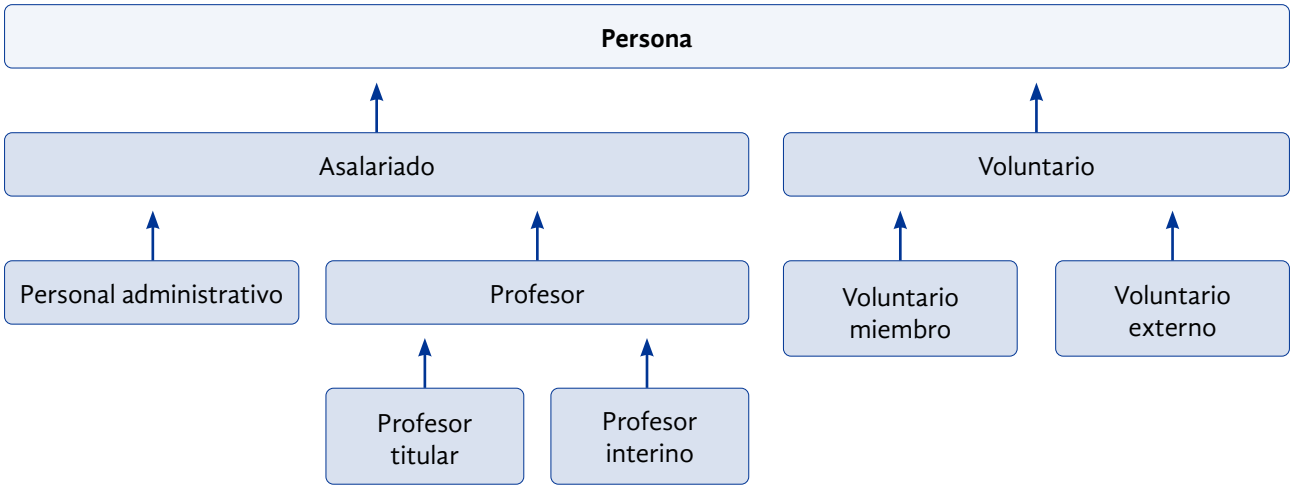

Fig.3. Herencia simple de clase Persona.

3.2. Herencia múltiple

Hay varios tipos de herencia, y entre ellos, podemos destacar la herencia múltiple.

En la herencia múltiple, una clase va a poder heredar de varias clases, es decir, **va a tener varias clases padre, y heredará el comportamiento de todas y cada una de ellas**.

Hay varios lenguajes de programación que soportan herencia múltiple, como, por ejemplo, C++, Python, Perl... Sin embargo, Java no es uno de ellos.

En Java, se emplea el uso de interfaces para lograr un comportamiento similar al de la herencia múltiple, pero con restricciones adicionales.

Este tipo de herencia es muy compleja, ya que puede contener **ambigüedades** en su definición. Esto puede venir motivado porque las herencias no se hayan definido en el orden correcto.

Un ejemplo de ambigüedades puede ser que haya dos clases de las que se hereda que tengan un atributo o un método que se llamen igual, entonces, si queremos llamar a ese atributo o método, ¿a cuál nos referimos?

¡! Sabías que...

En el contexto de la herencia múltiple, el 'diamante de la muerte' es un problema que surge cuando una clase hereda de dos clases que a su vez heredan de una misma clase.

La solución a este problema debe aportarla cada compilador de cada lenguaje de programación, es decir, probablemente C++ no lo solucionará de la misma forma que lo haga Python.

Concretamente, en C++, para solucionar este problema, el compilador nos obliga a indiciar el nombre de la clase padre a la que pertenece el atributo o método que queremos utilizar.

Como vemos, el uso de la herencia múltiple requiere un comportamiento que no se va a obtener ni a tratar a lo largo de estas unidades, por lo que, como en Java no se implementa la herencia múltiple, no lo vamos a utilizar.

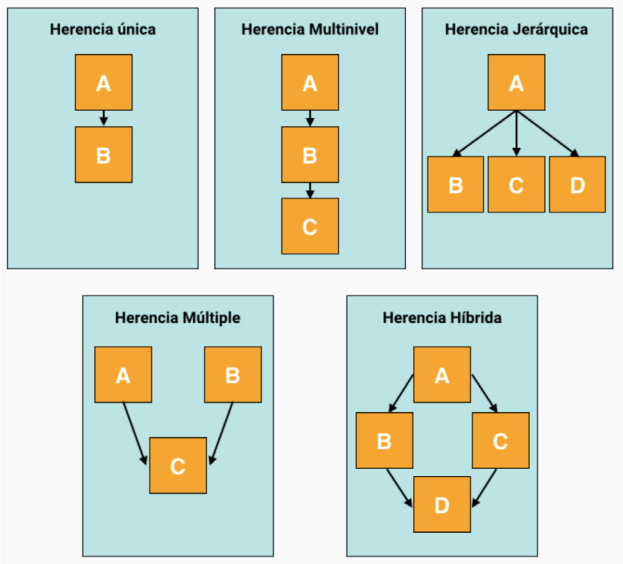

Fig.4. Tipos de herencia.

/ 4. Caso práctico 1: "Hasta dónde llegar con la herencia"

Planteamiento: La herencia en la programación orientada a objetos es un concepto fundamental que permite a una clase (subclase o clase derivada) heredar las propiedades y los métodos de otra clase (superclase o clase base). Sin embargo, hay límites y consideraciones importantes sobre hasta dónde llegar con la herencia.

Nudo: Se está analizando el código de un proyecto en el que se han creado varias clases que heredan de otra, pero que no tienen funcionalidad ninguna, es decir, no tienen nuevos atributos ni nuevos métodos, solo heredan. ¿Qué piensas al respecto? ¿Crees que una clase que hereda de otra, pero que no aporta funcionalidad ninguna, tiene sentido en un proyecto?

Desenlace: Cuando estamos aprendiendo el concepto de herencia, podemos caer en este fallo. Si repasamos el concepto de herencia visto anteriormente, una clase tiene sentido que herede de otra cuando agrega alguna funcionalidad nueva, tanto atributos como métodos. Se decía entonces que esta es un tipo de la clase que hereda.

Si, por el contrario, creamos una clase que hereda de otra sin que aporten ninguna nueva funcionalidad, no estamos aplicando bien el concepto de herencia, ya que esas clases no tienen un objetivo propio dentro del programa.

También puede ser que no las hayamos definido bien y se nos haya olvidado algún detalle para que les proporcione algún tipo de funcionalidad.

En cualquier caso, si tenemos una clase que hereda de otra y no aporta ninguna funcionalidad, lo único que podemos hacer con ella es eliminarla.

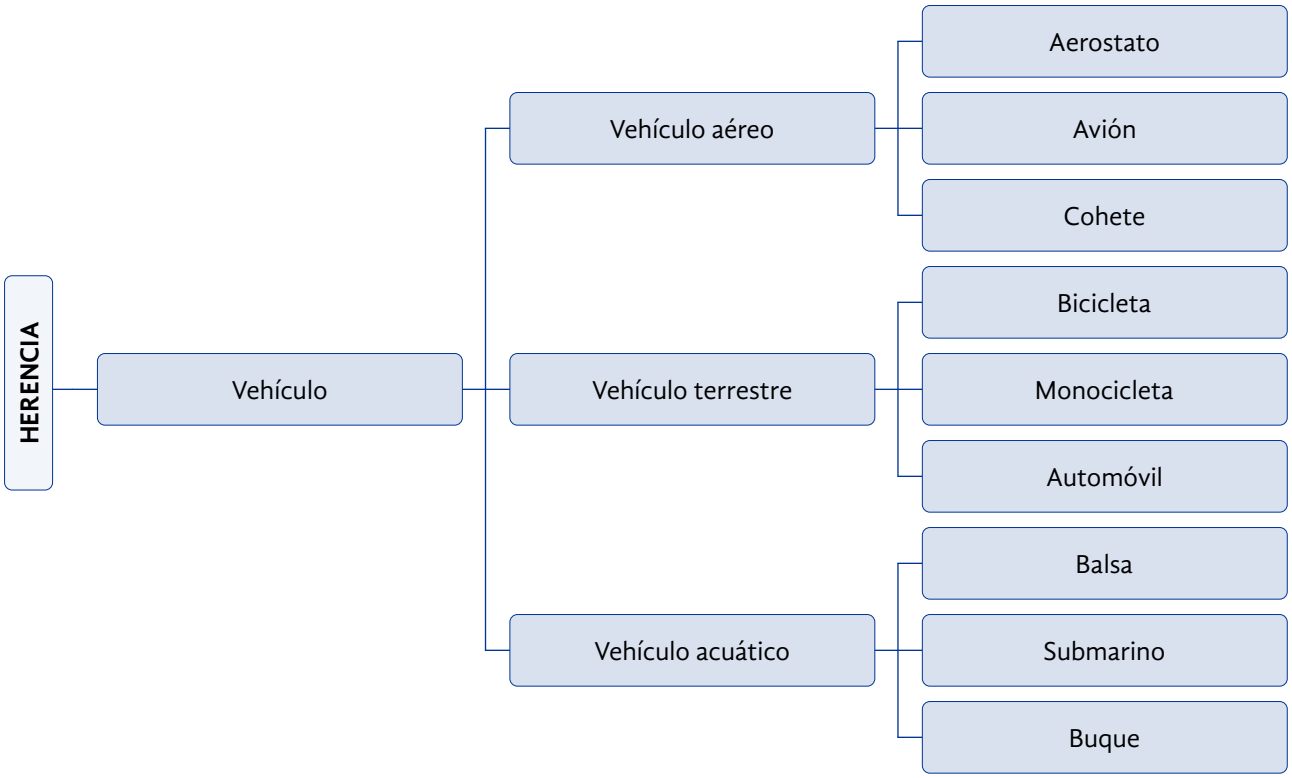

Fig.5. Utilidad de la herencia.

/ 5. Modificador de acceso para herencia

Con el uso de la herencia, la clase hija hereda los atributos y métodos de la clase padre, pero surge una cuestión: ¿hereda todos los atributos?; y, en caso contrario, ¿qué métodos y comportamientos heredan de la clase padre?

Hasta ahora tenemos tres modificadores de visibilidad y con el uso de la herencia surge un nuevo modificador: **protected**.

- **public**: Los miembros públicos son accesibles desde cualquier clase en cualquier paquete.
- **Sin modificador**: Los miembros sin un modificador de acceso, a menudo llamados 'paquete privado', son accesibles solo dentro del mismo paquete. Si una clase hereda de otra en el mismo paquete, los miembros sin modificador de acceso son heredados y pueden ser accedidos por la clase heredada. Sin embargo, si una clase hereda de otra en un paquete diferente, los miembros sin modificador de acceso no serán heredados ni accesibles en la clase derivada.
- **private**: Los miembros privados son accesibles solo dentro de la clase en la que se declaran. Los miembros privados no son heredados por las clases hijas y, por lo tanto, no son accesibles en ellas.
- **protected**: Los miembros protegidos son accesibles dentro del mismo paquete y por las clases derivadas (subclases), ya estén en el mismo paquete o no. Los miembros protegidos pueden ser heredados por las subclases y pueden ser accedidos directamente por ellas.

A continuación se muestra una tabla resumen de los modificadores de visibilidad:

	Propia clase	Clase vecina	Subclase	Clases externa
private	Visible	No visible	No visible	No visible
Sin modificador	Visible	Visible	No visible	No visible
public	Visible	Visible	Visible	Visible
protected	Visible	Visible	Visible	No visible

Tabla 1. Modificadores de visibilidad.

Dada la clase A de la siguiente figura, define el código de las siguientes variables enteras.

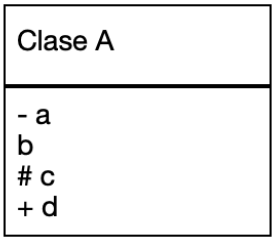

Fig.6. UML clase A.

```
class A {
        private int a; // visible unicamente en la clase
        int b;// visible en el paquete
        protected int c; // visible en el paquete y las clases que hereden aunque sean externas
        public int d; //visibilidad total
}
```

En el siguiente audio podrás aprender más sobre el apartado anterior.

Audio 1. "Recomendación con protected"
https://bit.ly/3jIYiks

/ 6. Sobreescritura de miembros heredados

Al heredar de una clase, sabemos que heredamos todos los métodos que funcionarán de la misma forma que en la clase padre, pero **¿y si por algún motivo quisiéramos cambiar el comportamiento, o simplemente añadirle alguna nueva funcionalidad a esos métodos?**

Esto es posible hacerlo gracias a la sobreescritura de métodos. Esta funcionalidad va ligada a la herencia, ya que **si por cualquier motivo el funcionamiento de un método en una clase hija varía, este se pueda implementar**.

En una clase pueden ser sobrescritos todos los métodos, salvo los constructores, pues estos han de llamarse dentro de los constructores de la clase hija.

Un método puede ser sobrescrito en una clase una vez, lo que no impide que si otra clase hereda de la clase que tiene el método sobrescrito, esta pueda volver a sobrescribirlo sin problema.

Para poder sobrescribir un método, como es lógico, primero tiene que estar implementado en la clase padre, posteriormente, debemos volver a crearlo con el mismo nombre, mismo tipo devuelto, y mismos parámetros en la clase hija, solo que ahora tendremos que indicar la siguiente sentencia en la parte superior: **@override**.

Con esta sentencia estamos indicando al compilador que ese método está sobrescrito y que cuando se llame desde un objeto de la clase que lo ha sobrescrito, deberá tener esa funcionalidad, no la de la clase padre.

```java
class Animal {
    public void hacerSonido() {
        System.out.println("Haciendo un sonido genérico.");
    }
}

class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("Ladrando...");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal animal = new Animal();
        animal.hacerSonido(); // Salida: Haciendo un sonido genérico.

        Animal perro = new Perro(); // Polimorfismo
        perro.hacerSonido(); // Salida: Ladrando...
    }
}
```

Fig.7. Sobreescritura de métodos.

En el ejemplo de la imagen previa, la clase Perro sobrescribe el método hacerSonido() de la clase Animal para imprimir 'Ladrando...' en lugar del mensaje genérico. Al crear una instancia de Perro y llamar al método hacerSonido(), se ejecuta la versión definida en la clase Perro.

Vídeo 1. "Ejemplo de sobreescritura de métodos"

https://bit.ly/31596RR

/ 7. Instrucción super

Cuando estamos utilizando la herencia y heredamos de una clase, ¿qué ocurre con los constructores?

Los constructores se heredan de forma indirecta, es decir, se heredan, pero no se pueden utilizar de forma directa con el operador new, sino que se deben crear los constructores propios de la clase hija.

Como no podemos acceder a la parte privada de la clase para poder iniciarla con los valores oportunos, debemos hacerlo con la instrucción super.

La instrucción super **permite llamar a las variables, los métodos y el constructor de la clase padre**.

7.1. Llamada al constructor de la clase padre

Al definir el constructor de la clase hija, lo primero que deberemos hacer de forma obligatoria es llamar al constructor de la clase padre utilizando super, y después de ello ya podremos escribir el código que sea necesario. En caso de que no llamemos a super en primer lugar, el compilador nos mostrará un fallo.

Esta llamada la podremos hacer en cualquiera de los constructores que hemos estudiado en unidades anteriores, pudiendo llamar a super con los parámetros que tenga el constructor al que queramos llamar (imagen):

```
class Padre {
    int edad;
    Padre(int edad) {
        this.edad = edad;
    }
}

class Hijo extends Padre {
    int altura;
    Hijo(int edad, int altura) {
        super(edad); // Llama al constructor de la clase Padre
        this.altura = altura;
    }
}
```

Fig.8. Constructor super().

7.2. Acceso a variables y métodos

Dentro de la clase hija, podemos utilizar super para acceder a variables y métodos de la clase padre, como se muestra en la imagen:

```
class Padre {
    int edad = 50;
    void saludar() {
        System.out.println("Hola desde la clase Padre");
    }
}

class Hijo extends Padre {
    void mostrar() {
        System.out.println(super.edad); // Accede a la variable edad de la clase Pad
        super.saludar(); // Llama al método saludar() de la clase Padre
    }
}
```

Fig.9. Acceso a variables y métodos de la clase padre.

Vídeo 2. "Creación de constructores de una clase heredada"

https://bit.ly/2OWuums

/ 8. Selección dinámica de métodos: Polimorfismo

El polimorfismo es otro de los pilares de la programación orientada a objetos y va ligado de forma muy estrecha a la herencia.

En Java, para poder utilizar el polimorfismo, debemos tener una serie de objetos que tengan una raíz común, es decir, **que hereden de la misma clase padre**.

El hecho de que una serie de objetos hereden de una misma clase padre va a traducirse en que van a ser **compatibles**, ya que, en definitiva, todos 'son un objeto' de la misma clase padre.

Cuando utilizamos polimorfismo, vamos a poder enviar exactamente el mismo mensaje a varios objetos, ya que serán compatibles con él al heredar de la misma clase.

El polimorfismo se puede clasificar en **dos grandes clases**:

- **Polimorfismo estático (polimorfismo ad hoc)**: En este tipo de polimorfismos es necesario definir de manera explícita el tipo de datos con el que se va a trabajar antes de ser utilizado.

- **Polimorfismo dinámico (polimorfismo paramétrico)**: En este tipo de código no se incluye el tipo de dato con el que se va a trabajar, pudiendo, de esa manera, emplear cualquier tipo de datos compatible.

Fig.10. Tipos de polimorfismo.

Si partimos de las definiciones previas de polimorfismo estático y dinámico, podría hacerse otra **clasificación basada en**:

- **Sobrecarga**: Capacidad de definir múltiples métodos en una clase con el mismo nombre, pero con diferentes listas de parámetros. La sobrecarga se resuelve en tiempo de compilación en Java, donde el compilador determina qué versión del método sobrecargado debe ser invocada en función de la lista de parámetros proporcionada en la llamada al método.

- **Sobreescritura**: Se refiere a la capacidad de una clase hija (subclase) de proporcionar una implementación específica para un método que ya está definido en su clase padre (superclase). Esto permite modificar el comportamiento de un método heredado para adaptarse a las necesidades de la clase hija. Para que la sobreescritura funcione correctamente, el método en la subclase ha de tener la misma firma (nombre, número y tipo de parámetros) que el método en la superclase.

- **Enlace dinámico**: Se refiere al proceso en el que se determina la implementación específica de un método durante el tiempo de ejecución, en lugar de en tiempo de compilación.

/ 9. Polimorfismo estático

Al declarar un objeto en Java, lo hacemos de la siguiente forma:

```
Tipo nombreobjeto = new Tipo( );
```

Donde se distinguen las siguientes partes:

- El **tipo**, que es el nombre de la clase que va a instanciar nuestro objeto. Esta va a ser la parte estática.

- El **nombre** de la clase a la que va a apuntar nuestro objeto en tiempo de ejecución. Esta va a ser la parte dinámica.

La idea consiste en que tanto el tipo del objeto como el tipo de la clase con la que instanciamos nuestro objeto van a ser los mismos, es decir, si el objeto es de tipo Cuadrado, en el new también tendremos Cuadrado y, por tanto, la asignación de tipos se va a realizar en tiempo de compilación.

La consecuencia más notable de esta declaración de tipos en tiempo de compilación es que no podremos llamar a métodos de clases que no sean del mismo tipo (aunque hayan heredado de la misma clase padre), es decir, que sean compatibles.

```
class Calculadora {
        static void multiplica(int a, int b){
                System.out.println("Resultado = "+ a*b);
        }
        static void multiplica(double a, double b){
                System.out.println("Resultado = "+ a*b);
        }
        static void multiplica(double a, double b, double c){
                System.out.println("Resultado = "+ a*b*c);
        }
}
public class Main {
        public static void main(String[] args) {
        // usando el primer método
        Calculadora.multiplica (3,5);
        // usando el segundo método
        Calculadora.multiplica (3.5,5.1);
        // usando el tercer método
        Calculadora.multiplica (3.6,5.2, 6.3);
        }
}
```

Como se puede observar, no es necesario instanciar un objeto de la clase Calculadora siguiendo el siguiente formato:

```
Tipo.metodo();
```

En la siguiente figura se puede ver un ejemplo de uso con la clase Math:

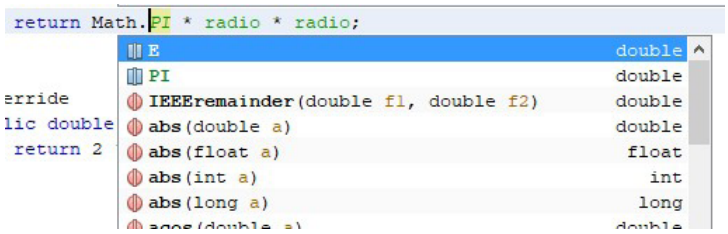

Fig.11. Clase Math.

/ 10. Polimorfismo dinámico

Si partimos de la declaración de un objeto en Java, hasta ahora se realizaba de la siguiente forma:

```
Tipo nombreobjeto = new Tipo( );
```

La idea es **que el tipo de objeto y el tipo de clase con la que instanciamos nuestro objeto deja de ser la misma**, es decir, si el objeto es de tipo Animal, en el new podemos tener una clase que herede, por ejemplo, Perro, y, por tanto, la asignación de tipos se va a realizar en tiempo de ejecución.

Cuando usamos polimorfismo dinámico, el compilador no puede decidir en tiempo de compilación a qué métodos se van a llamar, sino que se tomará esa decisión en tiempo de ejecución, con la llamada vinculación dinámica. Estas funciones polimórficas también se denominan **funciones virtuales**.

En este tipo de polimorfismo, la consecuencia más significativa de esta declaración de tipos en tiempo de ejecución es que sí podremos llamar a métodos de clases que no sean del mismo tipo, siempre que hayan heredado de la misma clase padre, es decir, que sean **compatibles**.

Es muy importante diferenciar que es el compilador el que verifica que los tipos sean correctos, pero cuando hacemos un cambio de tipo dinámico, lo hace la propia máquina virtual de Java, consiguiendo así que los tipos se puedan cambiar de forma dinámica en la ejecución del programa.

En la programación orientada a objetos, un **método virtual** es aquel que puede ser sobrescrito en una clase derivada para proporcionar una implementación específica para esa clase derivada. Esto permite que el método se comporte de manera diferente según el tipo de objeto en tiempo de ejecución, lo que es esencial para lograr el polimorfismo.

En Java, para definir un método como virtual, simplemente lo declaramos como público o protegido en la clase base, y luego lo sobrescribimos en la clase derivada utilizando la palabra clave @Override. Esto indica al compilador que estamos redefiniendo un método de la clase base.

```java
class Animal {
    public void hacerSonido() {
        System.out.println("El animal hace un sonido");
    }
}

class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("El perro ladra");
    }
}

class Gato extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("El gato maulla");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal animal1 = new Perro();
        Animal animal2 = new Gato();

        animal1.hacerSonido(); // Output: El perro ladra
        animal2.hacerSonido(); // Output: El gato maulla
    }
}
```

Fig.12. Polimorfismo dinámico.

En el siguiente recurso podrás aprender más sobre el apartado anterior.

¡! Sabías que...

El polimorfismo dinámico es conocido como polimorfismo de anulación, ya que se produce cuando un método en una subclase 'anula' un método de la superclase con la misma firma (nombre y tipo de parámetros).

/ 11. Caso práctico 2: "Salario del empleado"

Planteamiento: Cuando se crea una jerarquía de herencia, es crucial identificar y colocar los atributos y métodos comunes en la clase padre para maximizar la reutilización de código y mantener un diseño limpio y modular.

Nudo: Vamos a crear un sistema simple para gestionar diferentes tipos de empleados en una empresa. Habrá una clase base llamada Empleado y dos clases derivadas: EmpleadoTiempoCompleto y EmpleadoMedioTiempo.

Ambos empleados tienen nombre, edad y un salario base. Todos los empleados tienen un método calcularSalario() que devuelve el salario, sin embargo, en el caso de ser empleado a tiempo completo, al salario base se le suma un bono, y en el caso del empleado a medio tiempo, tiene una tarifa por horas trabajadas y el número de horas trabajadas.

Implementa ambas clases junto a su método calculaSalario().

Desenlace: Partimos de la clase Empleado que tiene atributos comunes a ambos tipos de empleados y un método calcularSalario que retorna el valor del salario base como muestra la siguiente imagen:

```java
public class Empleado {
    private String nombre;
    private int edad;
    private double salarioBase;

    public Empleado(String nombre, int edad, double salarioBase) {
        this.nombre = nombre;
        this.edad = edad;
        this.salarioBase = salarioBase;
    }

    public double getSalarioBase() {
        return salarioBase;
    }


    public double calcularSalario() {
        return salarioBase;
    }
```

Fig.13. Clase Empleado.

A partir de aquí, será necesario construir dos clases (imagen) que hereden de Empleado con los atributos específicos de cada una de ellas y el método calcularSalario sobrescrito.

```java
public class EmpleadoTiempoCompleto extends Empleado {
    private double bono;

    public EmpleadoTiempoCompleto(String nombre, int edad, double salarioBase, double bono) {
        super(nombre, edad, salarioBase);
        this.bono = bono;
    }

    @Override
    public double calcularSalario() {
        return getSalarioBase() + bono;
    }

public class EmpleadoMedioTiempo extends Empleado {
    private int horasTrabajadas;
    private double tarifaPorHora;

    public EmpleadoMedioTiempo(String nombre, int edad, double salarioBase, int horasTrabajadas, double tarifaPorHora) {
        super(nombre, edad, salarioBase);
        this.horasTrabajadas = horasTrabajadas;
        this.tarifaPorHora = tarifaPorHora;
    }

    @Override
    public double calcularSalario() {
        return getSalarioBase() + (horasTrabajadas * tarifaPorHora);
    }
```

Fig.14. Clases Empleado a tiempo completo y a medio tiempo.

/ 12. Métodos heredados

En unidades anteriores, hemos visto que las clases tenían dos tipos de métodos que nos permitían obtener y modificar los valores de las variables privadas:

- **Observadores**: Estos métodos permiten observar el valor de una variable privada. Son los métodos **get**.
- **Modificadores**: Estos métodos permiten modificar el valor de una variable privada. Son los métodos **set**.

Al heredar de una clase ya sabemos que obtenemos todo su comportamiento, métodos incluidos, pero ¿qué ocurre con los métodos observadores y modificadores?

Estos métodos también se heredan, como métodos que son, así que en la clase hija no habría que volver a implementarlos.

No obstante, hay que tener cuidado porque los métodos observadores y modificadores de las variables nuevas de la clase hija sí que hay que implementarlos, y cuando vayamos a usar un objeto de la clase, veremos que podremos invocar tanto a los métodos observadores y modificadores de las variables de la clase como a los de su clase padre.

Con esto, observamos nuevamente que la herencia nos reduce de forma significativa el código que debemos escribir en nuestras clases.

Una **mala práctica** sería el volver a definir los métodos observadores y modificadores de la clase padre en la clase hija y llamarlos con super. Esto no tendría ningún sentido, ya que no hará más que complicar la implementación de la clase de forma innecesaria.

```
public class Punto3D extends Punto2D {
    int z;

    public Punto3D(int x, int y, int z) {
        super(x, y);
        this.z = z;
    }

    public int getZ() {
        return z;
    }

    public void setZ(int z) {
        this.z = z;
    }

}
```

Fig.15. Ahorro de código.

/ 13. Clases y métodos finales

En Java, podemos utilizar las palabras clave final tanto para clases como para métodos en el contexto de la herencia.

13.1. Clases Finales

Una clase Final no puede tener subclases. Es decir, **no se puede heredar de una clase Final**. Esto significa que no se puede extender una clase Final. Su definición se muestra en la imagen:

```
final class ClaseFinal {
    // Contenido de la clase final
}
```

Fig.16. Clase Final.

13.2. Métodos finales

Un método final no se puede sobrescribir en las subclases. Esto significa que **la implementación del método final en la clase padre es definitiva y no se puede cambiar en las clases hijas**.

```
class Padre {
    final void metodoFinal() {
        // Contenido del método final
    }
}

class Hijo extends Padre {
    // No se puede sobrescribir metodoFinal()
    // Intentar sobrescribirlo resultará en un error de compilación
}
```

Fig.17. Método final.

13.3. Uso y beneficios

- **Seguridad y estabilidad**: Al marcar una clase o método como final, estamos indicando que su implementación no debe ser modificada. Esto puede ayudar a garantizar la estabilidad y seguridad del código.

- **Optimización**: Los compiladores pueden realizar optimizaciones en métodos finales, ya que saben que no serán sobrescritos. Esto puede conducir a un mejor rendimiento en algunos casos.

- **Control de diseño**: A veces, queremos evitar que una clase o método sea modificado o extendido por razones de diseño. Marcarlos como final proporciona un control claro sobre su comportamiento y uso.

Es importante tener en cuenta que si marcamos una clase como final, todos sus métodos son implícitamente finales y no pueden ser sobrescritos. Sin embargo, si marcamos un método como final en una clase, solo ese método específico no puede ser sobrescrito en las subclases.

En resumen, las clases y los métodos finales en Java proporcionan una forma de garantizar que ciertos aspectos de nuestro código no sean modificados o extendidos, lo que puede ser útil para mantener la coherencia y seguridad en nuestra aplicación.

/ 14. Resumen y resolución del caso práctico de la unidad

La importancia de comprender la herencia entre clases radica en su capacidad para fomentar la reutilización de código, permitiendo que las subclases hereden atributos y métodos de las superclases, lo que es fundamental para la organización eficiente del código y el desarrollo de jerarquías de clases coherentes y estructuradas.

Antes de heredar los atributos y métodos de las superclases, es importante definir su visibilidad y tener en cuenta que estos últimos (métodos) pueden ser sobrescritos para adaptar el comportamiento heredado, lo que es crucial para la personalización de las clases.

El siguiente punto importante es la selección dinámica de métodos en tiempo de ejecución, facilitada por la herencia, que es esencial para lograr el polimorfismo, lo que permite que un método se comporte de manera distinta en función del objeto al que pertenece.

Por último, se ha introducido el concepto de clase Final, empleado para restringir la herencia y evitar la modificación de ciertas clases, lo que contribuye a la integridad y seguridad del código.

Fig.18. Esquema resumen del tema.

Resolución del caso práctico de la unidad

En primer lugar, se definirá una clase base Animal con los atributos comunes a las clases Perro y Gatos (figura).

```
public class Animal {
    private String nombre;
    private int edad;
    private String raza;
    private double peso;

    public Animal(String nombre, int edad, String raza, double peso) {
        this.nombre = nombre;
        this.edad = edad;
        this.raza = raza;
        this.peso = peso;
    }
```

Fig.19. Clase Animal.

Una vez definida la clase Padre, se definen las clases Perro y Gato que heredan de Animal.

```
public class Gato extends Animal {
    private String tipoDePelaje;

    public Gato(String nombre, int edad, String raza, double peso, String tipoDePelaje) {
        super(nombre, edad, raza, peso);
        this.tipoDePelaje = tipoDePelaje;
    }

public class Perro extends Animal {
    private String nivelDeEntrenamiento;

    public Perro(String nombre, int edad, String raza, double peso, String nivelDeEntrenamiento) {
        super(nombre, edad, raza, peso);
        this.nivelDeEntrenamiento = nivelDeEntrenamiento;
    }
```

Fig.20. Clase Gato y Perro.

/ 15. Bibliografía

Bhargava, A. Y. (2019). *Algoritmos: Una guía ilustrada para programadores y curiosos.* Madrid: Ediciones Anaya Multimedia (Grupo Anaya).

Bloch, J. (2018). *Effective Java (3ª ed.).* Addison-Wesley.

Caro, C. M.; Ramos, A. N., & Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. PRENTICE HALL.

Knuth, D. E. (1997). *The Art of Computer Programming (Vol. 1: Fundamental Algorithms) (3ª ed.).* Addison-Wesley.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S. A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference (11ª ed.).* McGraw-Hill Education.